```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (
```

```
import nltk
```

```
Start coding or generate with AI.
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv("spam.csv", encoding='latin-1')
```

```
df.head()
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(5572, 5)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

```
df.isnull().sum()
```

	0
v1	0
v2	0
Unnamed: 2	5522
Unnamed: 3	5560
Unnamed: 4	5566

dtype: int64

```
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
```

```
df.rename(columns={
    'v1': 'Category',
    'v2': 'Message'
}, inplace=True)
```

```
df.head()
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df | New interactive sheet

```
df.duplicated().sum()
```

```
np.int64(403)
```

```
df.drop_duplicates(inplace=True)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.shape
```

```
(5169, 2)
```

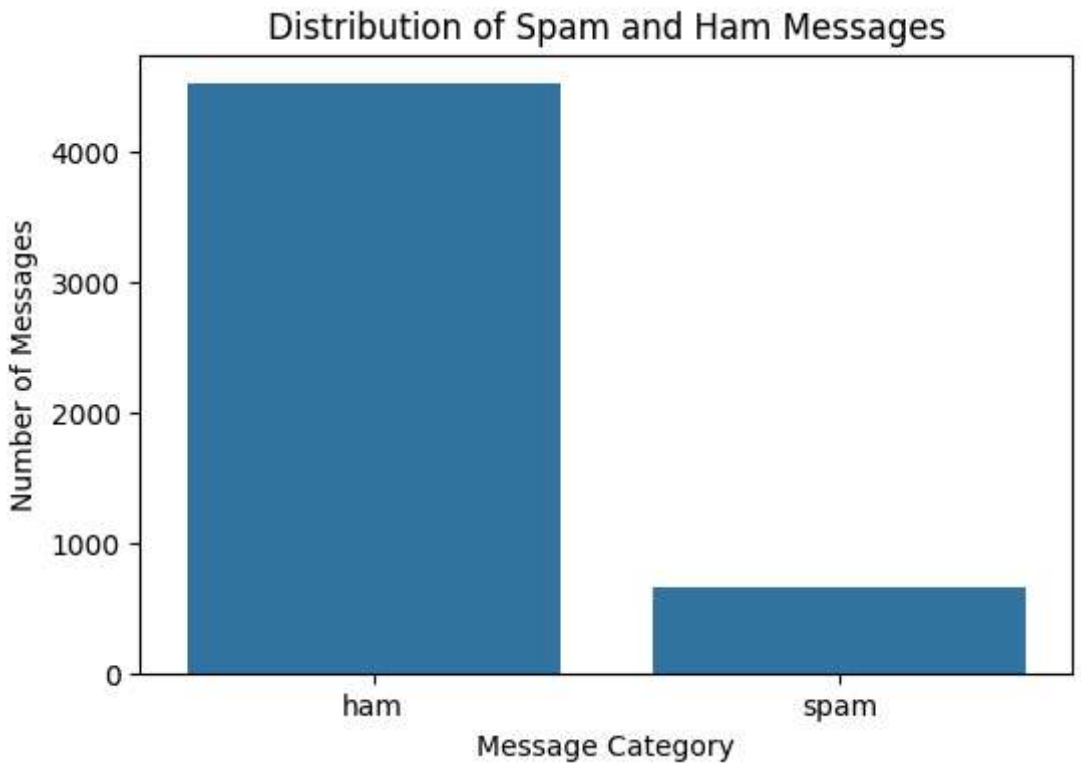
```
df['Category'].value_counts()
```

Category	count
ham	4516
spam	653

dtype: int64

```
plt.figure(figsize=(6,4))

sns.countplot(x='Category', data=df)

plt.title("Distribution of Spam and Ham Messages")
plt.xlabel("Message Category")
plt.ylabel("Number of Messages")
```

```
plt.show()
```




```
(df['Category'].value_counts(normalize=True) * 100).round(2)
```

Category	proportion
ham	87.37
spam	12.63

dtype: float64

```
df['num_characters'] = df['Message'].apply(len)
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
True
```

```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
from nltk.tokenize import word_tokenize
df['num_words'] = df['Message'].apply(lambda x: len(word_tokenize(x)))
```

```
from nltk.tokenize import sent_tokenize
df['num_sentences'] = df['Message'].apply(lambda x: len(sent_tokenize(x)))
```

```
df.head()
```

	Category	Message	num_characters	num_words	num_sentences
0	ham	Go until jurong point, crazy.. Available only ...	111	24	2
1	ham	Ok lar... Joking wif u oni...	29	8	2
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	ham	U dun say so early hor... U c already then say...	49	13	1

Next steps: Generate code with df | New interactive sheet

```
df[['num_characters', 'num_words', 'num_sentences']].describe()
```

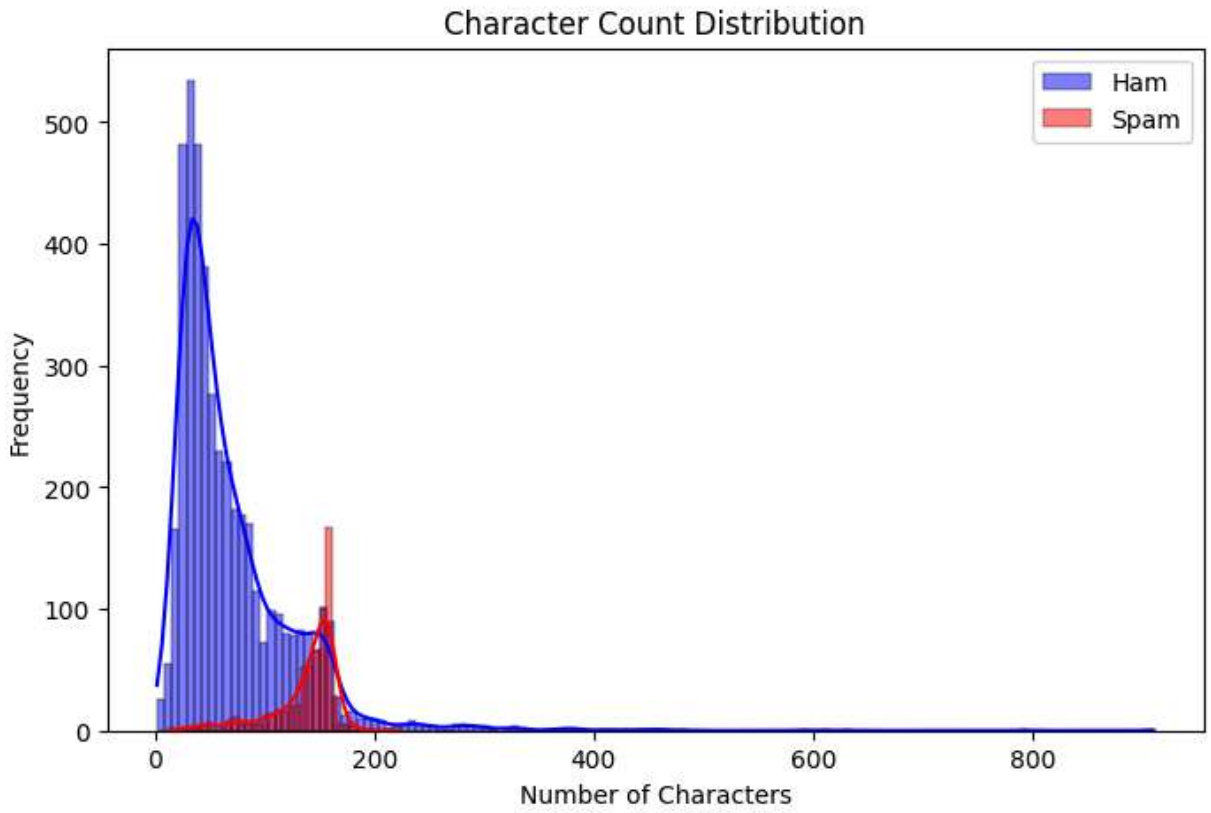
	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

```
df.groupby('Category')[['num_characters', 'num_words', 'num_sentences']].descri
```

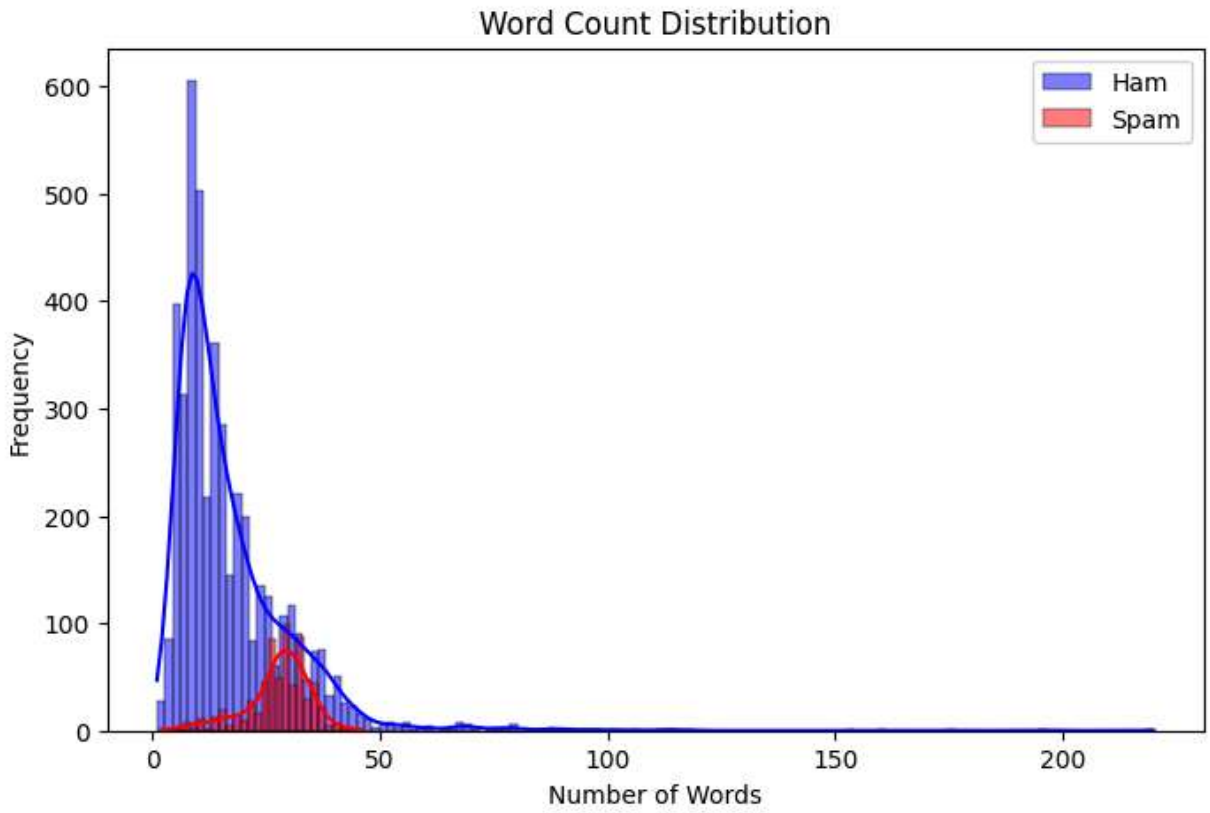
	num_characters							num_words	
	count	mean	std	min	25%	50%	75%	max	count	me
Category										
ham	4516.0	70.459256	56.358207	2.0	34.0	52.0	90.0	910.0	4516.0	17
spam	653.0	137.891271	30.137753	13.0	132.0	149.0	157.0	224.0	653.0	27

2 rows × 24 columns

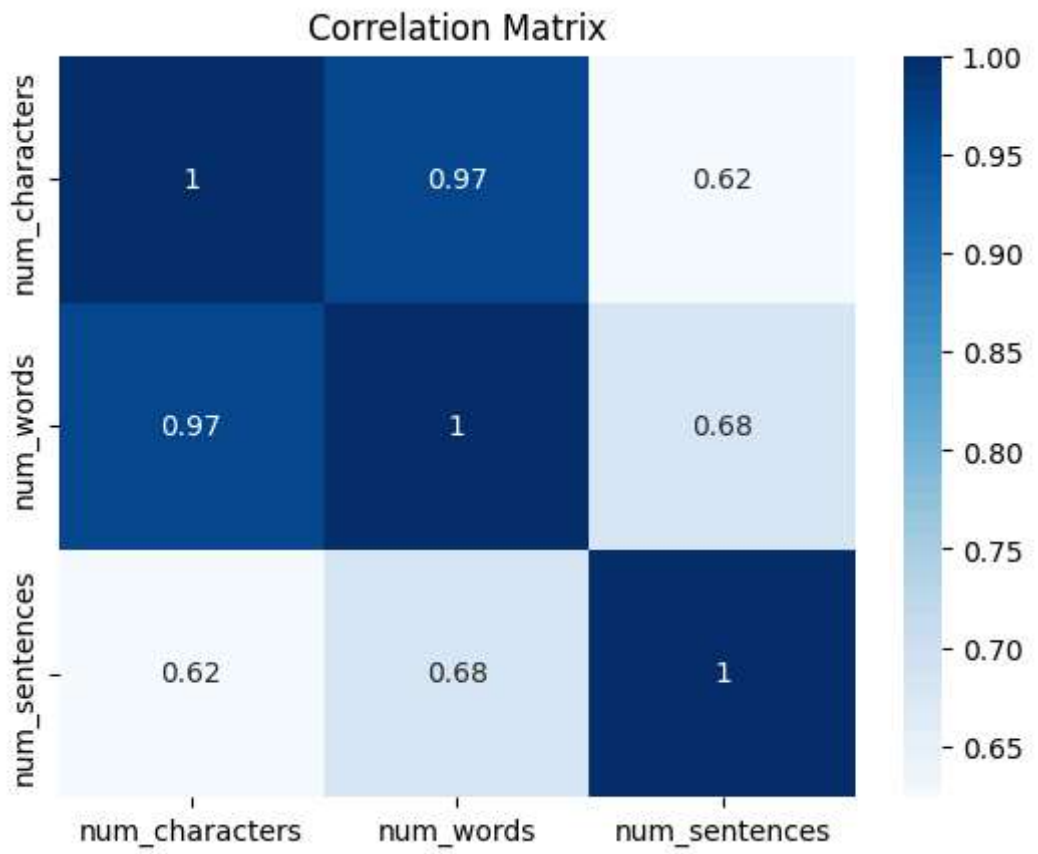
```
plt.figure(figsize=(8,5))

sns.histplot(df[df['Category'] == 'ham']['num_characters'],
             color='blue',
             label='Ham',
             kde=True)

sns.histplot(df[df['Category'] == 'spam']['num_characters'],
             color='red',
             label='Spam',
             kde=True)

plt.title("Character Count Distribution")
plt.xlabel("Number of Characters")
plt.ylabel("Frequency")
plt.legend()

plt.show()
```




```
plt.figure(figsize=(8,5))

sns.histplot(df[df['Category'] == 'ham']['num_words'],
             color='blue',
             label='Ham',
             kde=True)

sns.histplot(df[df['Category'] == 'spam']['num_words'],
             color='red',
             label='Spam',
             kde=True)

plt.title("Word Count Distribution")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.legend()

plt.show()
```




```
sns.heatmap(
    df[['num_characters', 'num_words', 'num_sentences']].corr(),
    annot=True,
    cmap='Blues'
)

plt.title("Correlation Matrix")
plt.show()
```




```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
import string
```

```
ps = PorterStemmer()
```

```
stopwords.words('english')[:20]
```

```
['a',
 'about',
 'above',
 'after',
 'again',
 'against',
 'ain',
 'all',
 'am',
 'an',
 'and',
 'any',
```

```
 'are',
 'aren',
 "aren't",
 'as',
 'at',
 'be',
 'because',
 'been']
```

```
string.punctuation
```

```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

```
def transform_text(text):

    text = text.lower()

    text = nltk.word_tokenize(text)

    y = []
    for i in text:
        if i.isalnum():
            y.append(i)

    text = y[:]
    y.clear()

    for i in text:
        if i not in stopwords.words('english') and i not in string.punctuation:
            y.append(i)

    text = y[:]
    y.clear()

    for i in text:
        y.append(ps.stem(i))

    return " ".join(y)
```

```
transform_text("Hi! How are you doing today?")
```

```
'hi today'
```

```
transform_text("Congratulations! You have WON a FREE prize worth ₹50,000.")
```

```
'congratul free prize worth'
```

```
df['transformed_text'] = df['Message'].apply(transform_text)
```

```
df[['Message', 'transformed_text']].head()
```

	Message	transformed_text
0	Go until jurong point, crazy.. Available only ...	go jurong point crazi avail bugi n great world...
1	Ok lar... Joking wif u oni...	ok lar joke wif u oni
2	Free entry in 2 a wkly comp to win FA Cup fina...	free entri 2 wkli comp win fa cup final tkt 21...
3	U dun say so early hor... U c already then say...	u dun say earli hor u c alreadi say
4	Nah I don't think he goes to usf, he lives aro...	nah think goe usf live around though

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(max_features=3000)
```

```
X = tfidf.fit_transform(df['transformed_text']).toarray()
```

```
X.shape
```

```
(5169, 3000)
```

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()
```

```
y = encoder.fit_transform(df['Category'])
```

```
y
```

```
array([0, 0, 1, ..., 0, 0, 0])
```

```
print("X Shape:", X.shape)
print("y Shape:", y.shape)
```

```
X Shape: (5169, 3000)
y Shape: (5169,)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train_text, X_test_text, y_train, y_test = train_test_split(
    df['transformed_text'],
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training Samples:", len(X_train_text))
print("Testing Samples:", len(X_test_text))
```

```
Training Samples: 4135
Testing Samples: 1034
```

```
X_train = tfidf.fit_transform(X_train_text)
```

```
X_test = tfidf.transform(X_test_text)
```

```
print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (4135, 3000)
X_test : (1034, 3000)
y_train: (4135,)
y_test : (1034,)
```

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s
```

```
mnb = MultinomialNB()
```

```
mnb.fit(X_train, y_train)
```

▾ MultinomialNB

```
MultinomialNB()
```

```
y_pred = mnb.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9700193423597679
```

```
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
```

```
Precision: 0.9807692307692307
```

```
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
```

```
Recall: 0.7786259541984732
```

```
f1 = f1_score(y_test, y_pred)
print("F1 Score:", f1)
```

```
F1 Score: 0.8680851063829788
```

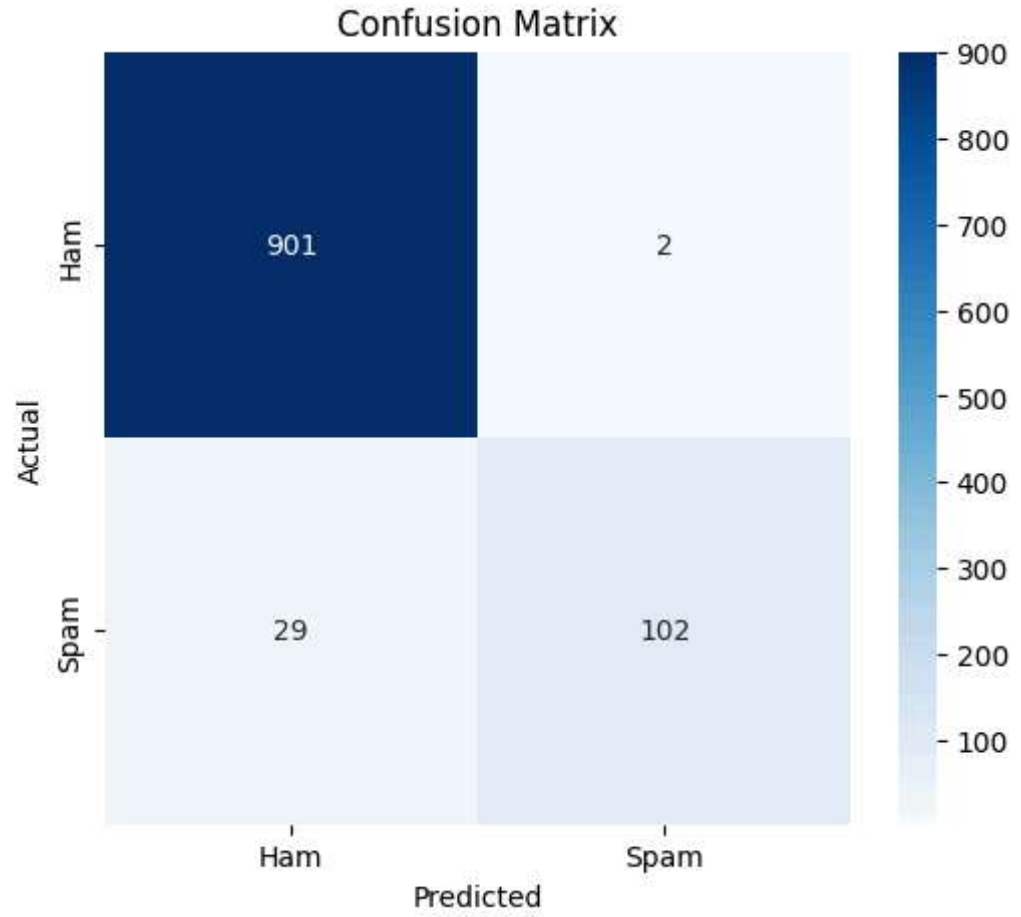
```
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Accuracy : 0.9700193423597679
Precision: 0.9807692307692307
Recall   : 0.7786259541984732
F1 Score : 0.8680851063829788
```

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[901   2]
 [ 29 102]]
```

```
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Ham', 'Spam'],
    yticklabels=['Ham', 'Spam']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```




```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.98       903
           1       0.98      0.78      0.87       131

    accuracy                           0.97      1034
   macro avg       0.97      0.89      0.93      1034
weighted avg       0.97      0.97      0.97      1034
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
lr = LogisticRegression(max_iter=1000, random_state=42)
svc = SVC(kernel='linear', probability=True, random_state=42)
dt = DecisionTreeClassifier(random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
knn = KNeighborsClassifier(n_neighbors=5)
```

```
def evaluate_model(model, X_train, X_test, y_train, y_test):

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    return accuracy, precision, recall, f1
```

```
results = []
models = {
    "Naive Bayes": mnb,
    "Logistic Regression": lr,
    "Support Vector Machine": svc,
    "Decision Tree": dt,
    "Random Forest": rf,
    "KNN": knn
}
for name, model in models.items():

    accuracy, precision, recall, f1 = evaluate_model(
        model,
        X_train,
        X_test,
        y_train,
        y_test
    )

    results.append([
        name,
        accuracy,
        precision,
        recall,
        f1
    ])
```

```
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
```

```
)

results_df
```

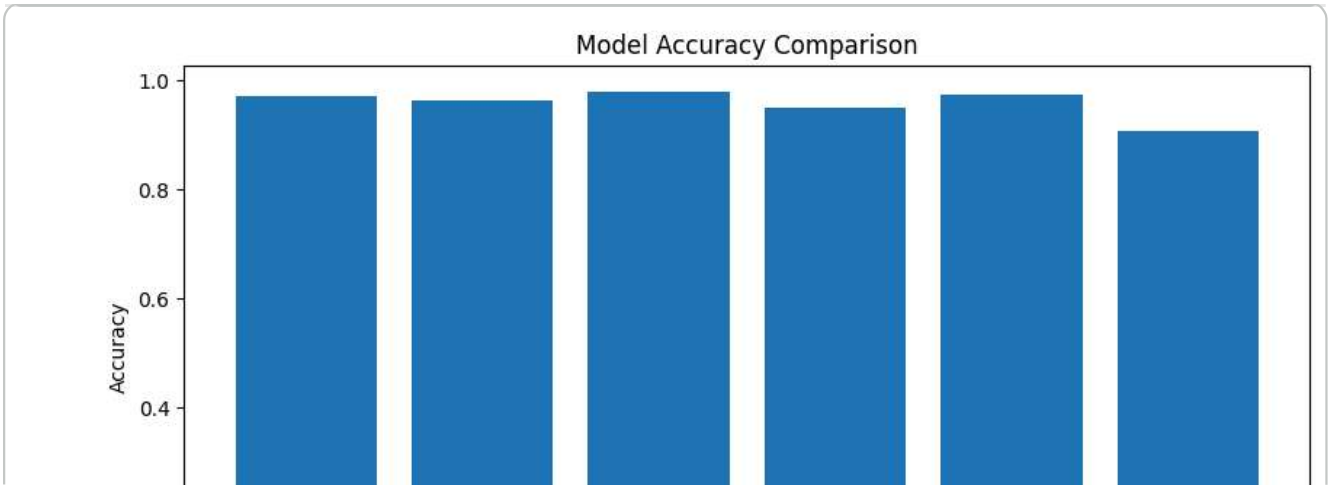
	Model	Accuracy	Precision	Recall	F1 Score
0	Naive Bayes	0.970019	0.980769	0.778626	0.868085
1	Logistic Regression	0.962282	1.000000	0.702290	0.825112
2	Support Vector Machine	0.978723	0.982301	0.847328	0.909836
3	Decision Tree	0.948743	0.830508	0.748092	0.787149
4	Random Forest	0.973888	0.981481	0.809160	0.887029
5	KNN	0.908124	1.000000	0.274809	0.431138

Next steps: Generate code with `results_df` New interactive sheet

```
results_df.sort_values(by="Accuracy", ascending=False)
```

	Model	Accuracy	Precision	Recall	F1 Score
2	Support Vector Machine	0.978723	0.982301	0.847328	0.909836
4	Random Forest	0.973888	0.981481	0.809160	0.887029
0	Naive Bayes	0.970019	0.980769	0.778626	0.868085
1	Logistic Regression	0.962282	1.000000	0.702290	0.825112
3	Decision Tree	0.948743	0.830508	0.748092	0.787149
5	KNN	0.908124	1.000000	0.274809	0.431138

```
plt.figure(figsize=(10,5))

plt.bar(results_df["Model"], results_df["Accuracy"])

plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")

plt.xticks(rotation=20)

plt.show()
```


Model Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4